

Proyecto Final

Nombre del proyecto: *[nombre-usuario]-final-project*

Tiempo estimado: 2 semanas

Vas a crear una API RESTful completa con Spring Boot que integre todo lo aprendido a lo largo del curso: CRUDs, relaciones complejas, seguridad con JWT, validaciones, pruebas, logging, control de versiones y despliegue con Docker.

Esta API gestionará usuarios, productos, pedidos y países, incluyendo relaciones entre ellos y restricciones de acceso según roles.

Requisitos funcionales

Repositorio GIT

Crea un repositorio Git para el proyecto y súbelo a un repositorio remoto (GitLab).

Haz commits frecuentes con mensajes descriptivos. Se recomienda practicar con ramas (*feature/**, *main*, etc.) simulando un flujo real de trabajo.

Entidades a implementar

User

- *id*
- *fullName*
- *email* → debe ser único
- *password* → debe estar encriptada (se recomienda BCrypt)
- *createdAt* → fecha de creación
- *isActive* → marca de borrado lógico (soft delete)
- *country* → referencia a un *Country* (relación many-to-one)

Un usuario puede tener varios pedidos (relación one-to-many con *Order*).

Country

- *code* → código de país (clave primaria)
- *name* → nombre del país

Un país puede estar asociado a muchos usuarios (relación one-to-many).

Product

- *id*
- *name*
- *price* → (float o decimal)
- *status* → estado del producto (por ejemplo: AVAILABLE, DISCONTINUED)
- *createdAt* → fecha de creación

Un producto puede estar en varios pedidos (relación many-to-many con *Order*, a través de *OrderProduct*).



Order

- `id`
- `user` → referencia al usuario que realiza el pedido (relación many-to-one)
- `status` → (ej. PENDING, COMPLETED, CANCELLED)
- `createdAt`

Relación con productos a través de una tabla intermedia (`OrderProduct`)

OrderProduct (tabla intermedia)

Representa la relación many-to-many entre `Order` y `Product`. Debe incluir:

- `orderId` → ID del pedido
- `productId` → ID del producto
- `amount`

Puedes modelarla con una clave compuesta o una entidad embebida.

Consideraciones técnicas

- Usa JPA y anotaciones adecuadas para modelar las relaciones.
- Las contraseñas deben guardarse en formato encriptado (BCrypt).
- Implementa seguridad con JWT y Spring Security:
 - Rol `ADMIN` puede acceder a todos los endpoints.
 - Rol `USER` tiene acceso limitado.
- Aplica validaciones usando Jakarta Validation.
- Usa `@ControllerAdvice` para el manejo de errores personalizados.
- Aplica el patrón DTO para exponer la información.
- Implementa borrado lógico mediante el campo `isActive`.

Endpoints obligatorios

Recurso	Operaciones	URL base
<code>User</code>	CRUD	<code>/users</code>
<code>Product</code>	CRUD + búsqueda filtrada	<code>/products</code>
<code>Order</code>	Crear, listar, actualizar	<code>/orders</code>
<code>Country</code>	CRUD	<code>/countries</code>
<code>Asignar un país</code>	PATCH	<code>/users/{id}/country</code>

El endpoint `GET /products` debe implementar filtros con `CriteriaBuilder` (mínimo dos criterios opcionales).



Acceso por roles

Endpoint	ADMIN	USER
/users	✓ Completo	✗ Sin acceso
/products	✓ Completo	✓ Solo GET con filtros
/orders	✓ Ve y modifica todos	✓ Solo ve/crea los suyos
/countries	✓ Completo	✓ Solo GET
/users/{id}/country	✓ Modifica cualquier usuario	✓ Solo modifica el suyo

Restricciones y validaciones especiales

Caso	Excepción	Código HTTP
Email duplicado al crear usuario	EmailAlreadyExistsException	409
Formato de email inválido (regex)	EmailNotValidException	422
Contraseña insegura (mín. 8 caracteres, etc.)	InvalidPasswordException	422
Nombre vacío al actualizar usuario	InvalidFullnameException	422
Usuario no encontrado al actualizar/eliminar	UserNotFoundException	404
USER intenta modificar el país de otro usuario	ForbiddenOperationException	403

El resto de validaciones (campos obligatorios, existencia de IDs, etc.) se consideran parte estándar del CRUD y no se detallan aquí.

Logging

- Usa logs informativos y de error donde consideres relevante.
- No satures la salida: loguea eventos clave (creación, errores, peticiones importantes).

Pruebas

- Aplica TDD siempre que sea posible.
- Cobertura mínima recomendada: **70%**
- Usa JUnit y Mockito para pruebas unitarias y de integración.
- Ejecuta SonarQube para verificar cobertura y calidad del código.



Dockerización

- Crea un `Dockerfile` para la aplicación.
- Usa `docker-compose.yml` para levantar la aplicación junto con la base de datos (por ejemplo PostgreSQL):

```
Unset
services:
  app:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - db

  db:
    image: postgres:latest
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: project
```

Cierre del proyecto

Asegúrate de:

- Tener una estructura de paquetes clara y coherente.
- Usar DTOs y servicios correctamente.
- Hacer commits frecuentes y bien comentados.
- Probar y validar todos los flujos principales.
- (Opcional) Documentar los endpoints con Swagger/OpenAPI.
- (Opcional) Usar ramas feature y pull requests para simular un entorno real.

🎉 ¡Enhorabuena por haber llegado hasta aquí!